Does Relative Point Support Improve Trust under Sparse-LiDAR Shift?

Project information

- **Track:** Safety and Evaluation
- **Maintainer:** Rohan Ohlan
- **Software:** Python, PyTorch, OpenPCDet, and scikit-learn; XGBoost is optional.
- **Hardware:** NVIDIA GeForce RTX 3060 Ti (8 GB), AMD Ryzen 7, and 32 GB RAM.

1. Summary

LiDAR detectors can produce confident but incorrect boxes when objects have few supporting points because of distance, occlusion, reduced sensor resolution, or corruption. This project asks whether a lightweight post-hoc monitor can identify those unreliable predictions without retraining the detector.

A frozen PointPillars model will produce Car detections on clean and artificially sparsified KITTI point clouds. The monitor will use detector confidence, range, predicted box geometry, and point support. Its main added feature is **relative point support**: the difference between the observed number of points in a predicted box and the number expected for a clean, correct detection at the same range and geometry.

The primary test is clean-to-sparse transfer. The monitor will be trained only on clean predictions and evaluated on clean data, random point dropout, and simulated 32-line and 16-line inputs. It succeeds only if it lowers the accepted-prediction error rate at 80% coverage compared with both native detector confidence and a strong post-NMS feature baseline.

2. Research question and scope

Research question

At the same candidate coverage, does relative point support rank incorrect PointPillars detections better under sparse-LiDAR shift than:

1. native confidence;
2. confidence plus range; and
3. a post-NMS LidarMetaDetect-style feature baseline (LMD-lite)?

Output

For each post-NMS candidate box, the monitor estimates

```
q_i = P(candidate i is correct | inference-time features)
r_i = 1 - q_i
```

A threshold on `q_i` accepts or flags each prediction. Ground truth is used to train and evaluate the monitor, but never as an inference-time input.

Novelty boundary

This project does **not** claim a new detector, a new use of range or point density, the first post-hoc LiDAR quality estimator, the first sparse-LiDAR study, or superiority over full LidarMetaDetect.

Its contribution is a controlled empirical test of one narrow idea:

> Does a clean-anchored, range- and geometry-conditioned point-support residual improve selective error ranking for a frozen, non-query LiDAR detector under sparse-input shift?

The main novelty is the clean-to-sparse evaluation and the physical interpretation of the residual, not a new model architecture. A negative result is valid if it shows that relative support adds no value beyond confidence, range, and ordinary density features.

3. Data and experimental split

Dataset and detector

- **Dataset:** KITTI 3D Object Detection
- **Detector:** PointPillars from OpenPCDet
- **Class:** Car
- **Detector metric:** KITTI `AP3D_R40` for Easy, Moderate, and Hard; Moderate is primary.

The detector checkpoint must have documented provenance. It is acceptable only if the final monitor-test frames were not used to train PointPillars.

Frame-level split

Use the standard OpenPCDet split of 3,712 detector-training frames and 3,769 held-out validation frames. Divide the 3,769 held-out frames into:

- 50% monitor training;
- 25% calibration and model selection; and
- 25% locked final testing.

Split by `frame_id`, never by individual detection. Every sparse version of a frame stays in the same partition. Commit the frame lists and random seeds.

Training regimes

- **Primary:** train and tune on clean frames, then test once on clean and sparse frames.
- **Corruption-aware:** train the trust classifier on a fixed mixture of clean and sparse training frames.
- **Cross-corruption:** train on random dropout and test on simulated line reduction, then reverse the direction.

The expected-support model is always fitted only on clean, correctly matched training predictions and is then frozen.

4. Sparse-input conditions

Random dropout

Retain each point independently with probabilities 0.75, 0.50, and 0.25. Generate deterministic seeds from `frame_id` and severity. Apply corruption before detector preprocessing.

Simulated line reduction

KITTI files do not contain reliable laser-ring identifiers, so this condition will be called **simulated scan-line reduction**, not hardware beam removal.

1. Compute each point's elevation angle.
2. Fit 64 ordered elevation centers using only clean monitor-training frames.
3. Freeze the centers and assign every point to its nearest center.
4. Keep alternating centers for the 32-line condition and every fourth center for the 16-line condition.
5. Report retained-point fraction, per-line occupancy, and elevation histograms.

If the public KITTI-64-to-32/16 transformation from prior cross-resolution work can be reproduced, it will be preferred and its source commit will be recorded.

5. Candidates, labels, and features

Fixed candidate set

Each method must rank the same predictions. For each input condition:

- run the detector once and save post-NMS Car predictions with score at least 0.01;
- use these predictions as the master candidate set; and
- evaluate nested score subsets at 0.05 (primary) and 0.10 (sensitivity).

Coverage is the accepted fraction of this fixed candidate set. Selective risk is the fraction of accepted candidates that are incorrect. These metrics evaluate prediction ranking; they are not KITTI AP after reordering detections.

Correctness labels

The primary label is correctness for Moderate-eligible Car detections. Predictions are processed in native-score order and matched one-to-one to unused eligible ground-truth boxes at `IoU3D >= 0.70`. Predictions associated with ignored objects or `DontCare` regions are excluded according to the selected OpenPCDet/KITTI evaluator.

A documented wrapper around the evaluator will generate TP, FP, and ignored labels. Synthetic tests will cover duplicate detections, ignored objects, and `DontCare` regions.

Features

The basic post-NMS features are:

- native confidence and clipped confidence logit;
- horizontal range;
- predicted length, width, height, volume, and yaw;
- point count in the oriented predicted box;
- log point count, zero-point indicator, and point density; and
- total frame point count and its ratio to the clean-training median.

The expected-support model estimates

```
m_hat_i = E[log(1 + point_count_i) | range_i, predicted_geometry_i]
relative_support_i = log(1 + point_count_i) − m_hat_i
```

The proposed logistic-regression monitor adds `relative_support` and predeclared interactions such as `confidence × relative_support` to the basic features. XGBoost may be used only as a model-capacity ablation.

6. Baselines and ablations

Method	Purpose
Native confidence	Main detector-ranking baseline
Temperature, positive-slope Platt, and isotonic scaling	Calibration baselines
Confidence plus range	Tests whether distance alone explains errors
LMD-lite logistic regression	Strong post-NMS baseline using confidence, range, geometry, point count/density, and frame support
Proposed logistic regression	Adds relative support and fixed interactions to LMD-lite
XGBoost, optional	Tests whether nonlinear model capacity matters

The critical ablation is **LMD-lite versus LMD-lite plus relative support**. This isolates the residual's incremental value after ordinary range and density information is already available.

Other ablations will compare:

- raw point count, density, and relative support;
- additive residual, support ratio, and standardized residual;
- relative support with and without global frame support;
- interactions included versus removed;
- clean-only, corruption-aware, and cross-corruption training; and
- matched test coverage versus a threshold frozen on clean calibration data.

Full LidarMetaDetect uses reflectance and pre-NMS proposal statistics. Unless those inputs and the complete method are reproduced, the baseline will be called **LMD-lite**, not LidarMetaDetect.

7. Evaluation

Primary endpoint

The primary endpoint is accepted-error rate at 80% fixed-candidate coverage under the simulated 32-line condition. For each main baseline, compute

```
Delta80 = risk_proposed(80%) − risk_baseline(80%)
```

Use 1,000 paired frame-bootstrap resamples and recompute the coverage cut in every resample. The primary claim succeeds only if the upper bound of the 95% confidence interval for `Delta80` is below zero against both native confidence and LMD-lite. Report the percentage-point difference and confidence interval.

Secondary metrics

- accepted-error rate at 60%, 80%, and 90% coverage;
- risk-coverage curves and AURC;
- error-detection AUROC and AUPRC;
- 15-bin correctness ECE, Brier score, and negative log-likelihood;
- KITTI `AP3D_R40`, recall, candidate count, and incorrect-label prevalence; and
- results for every corruption type and severity.

Calibration and ranking will be interpreted separately. Temperature scaling and positive-slope Platt scaling preserve score order and therefore cannot improve AURC. Isotonic regression can create ties, so its selective metrics will use tie-averaged evaluation.

Operational threshold transfer

Each method will also choose a threshold on the clean calibration split and freeze it. On each final-test condition, report achieved coverage, accepted risk, recall, and official KITTI `AP3D_R40`. Retained boxes keep their native-score order before official evaluation.

8. Interpretation, limitations, and reproducibility

The monitor evaluates emitted boxes; it cannot recover objects that PointPillars completely misses and does not provide a safety guarantee. Detector recall and false negatives will therefore be reported separately. Simulated line reduction is also not a substitute for evaluation on a real lower-resolution sensor.

If relative support helps, the conclusion will be limited to improved error ranking under the tested conditions. If it does not help, the analysis will test whether native confidence already encodes point support, whether raw density is sufficient, or whether the clean expectation model fails to transfer.

Save one row per candidate in CSV or Parquet, including candidate and frame IDs, split, corruption, native score, all monitor features, matched ground-truth ID, IoU, correctness labels, and ignore status. Record the OpenPCDet commit, detector configuration, checkpoint hash and provenance, split files, corruption seeds, package versions, candidate thresholds, matching code, and metric definitions.

9. Milestones

Weeks	Deliverable
1–2	Reproduce clean PointPillars results and verify checkpoint provenance

Weeks	Deliverable
3–4	Extract candidates and validate matching/ignore logic
5–6	Generate and validate sparse-input conditions
7–8	Run native-confidence, calibration, range, and LMD-lite baselines
9–10	Fit the relative-support monitor and run the locked primary test
11–12	Complete ablations, cross-condition tests, and confidence intervals
13–14	Analyze positive or negative results and finalize reproducibility instructions

References

See `literature_sota_survey.md` for the literature review and detailed baseline mapping.